ADDENDUM

Three evaluation layers for the emMorph Python port

Version 0.2, 18 October 2026. Farzaneh Velaeishekan. Supervised by Prof. Balázs Indig, Eötvös Loránd University. Prepared at K. N. Toosi University of Technology.

Contents

1. What Prof. Indig asked for
2. The three-layer architecture
3. How each layer is run
4. The word list: external, generated, fuzzed
5. Three weeks, 18 October to 8 November
6. The longer roadmap
7. The thirty-three files and the CI jobs
8. References

1. What Prof. Indig asked for

The v0.1 audit, dated 18 October 2026, stands as the historical record of the baseline. This addendum does not revise it. It records three additions from Prof. Indig's email of 17 October 2026. The sentences below are his, not a paraphrase.

On the word list

> “I think a sufficiently large word list from any Hungarian corpus would suffice. You should try to generate all possible word forms and may try fuzzing as well.”

The oracle of leaf-script stdout is necessary and not sufficient. A Hungarian word list, the closure of the lexicon under the legal tags, and a seeded set of Hungarian-like strings are three different probes. Section 4 says where each one comes from.

On intermediate state

> “But as the code is modular and creates intermediate states during the compilation mean that a comparison of the stability of these states across different compilations is sufficient to catch hash stability issues. See the drop-in replacement approach. + Immediate state comparison.”

The build writes about thirty files under gen/ before it writes a transducer. Two compilations that differ only in PERL_HASH_SEED must agree on those files after hash lines have been sorted. A file that moves is a bug in the script that wrote it. A file that is absent is a gap in the public tree, not a hash bug. The two outcomes are reported separately so that a missing humlex.pl is not mistaken for an unstable encode.pl.

On the user-facing analyser

> “As you want to create extrinsic evaluation I'd recommend using emmorphpy with the compiled FST. It has much nicer API from the end-user perspective.”

emmorphpy, at https://github.com/nytud/emmorphpy, is the program a linguist actually calls. Comparing its analyses on two transducers is the extrinsic test. Leaf scripts and gen/ files can match while the analyses a user sees still diverge. Layer 3 is that last comparison.

On the longer term

> “In the long run you should incorporate the two (emmorph and emmorphpy) as well as extending libhfst if it still do not support the optimised lookup feature in its Python API. As currently there are serious bottlenecks beacuse of the popen() approach.”

The spelling beacuse is his. The point is not. Section 6 splits the remark into three phases and says why a fresh process per lookup cannot stay.

2. The three-layer architecture

Each layer compares a different artefact. A failure at layer 1 stops the port of that script. A failure at layer 2 names the producer. A failure at layer 3 is what a dictionary user would notice. The layers are not substitutes. A stable gen/ directory does not prove that hfst-lookup returns the same cohort.

Layer	What it compares	What it catches	Tool
1. Leaf scripts	Perl stdout against the Python port, and seed 0 against seed 1.	NEW_BUG in a port, and hash-order bugs that survive normalisation.	harness/run_diff.py and harness/reproducibility_check.py
2. Intermediate state	Each gen/ file after xlx, built twice with different PERL_HASH_SEED values.	A producer whose output is unstable once hash lines are sorted.	harness/intermediate_state_diff.py

3. Extrinsic	Analyses of one word list on the Perl FST and on the Python FST.	Divergent analyses that no intermediate file would show.	harness/extrinsic_eval.py via emmorphpy or hfst-lookup

Table 1. The three layers, what each one diffs, and which command runs it.

Normalisation is shared. harness/normalize.py rewrites a Perl hash assignment so that its pairs are ordered by key. Layer 1 uses that on stdout. Layer 2 uses the same function on every text file in gen/ before sha256. A pure reordering of %mcat is therefore not reported as unstable. A change of a value, or a missing key, is.

Classification of a leaf-script diff stays in harness/classify.py. OLD_BUG means the Perl output is the one that is wrong. NEW_BUG means the Python output is wrong. INTENTIONAL_CHANGE is a difference the manifest has already signed. Layer 2 does not use those labels. Its vocabulary is STABLE, UNSTABLE, and MISSING, because the question there is whether two seeded builds agree, not whether Python matches Perl.

3. How each layer is run

Layer 1, leaf scripts

The differential harness reads harness/manifest.yaml. For each entry it prefers the frozen oracle under oracle/ and runs the Python module in the Perl tree so that paths such as src/sfx.1 resolve to the same file. PERL_HASH_SEED is fixed when the oracle is generated.

```
python harness/generate_oracle.py --perl-dir emmorph-perl --out oracle/

python harness/run_diff.py --manifest harness/manifest.yaml --perl-dir emmorph-perl

python harness/reproducibility_check.py --perl-dir emmorph-perl
```

A module is done when run_diff.py exits 0. That means there is no NEW_BUG and no unclassified hunk. getsfxtags, bsort, lcase, sort, and uniq are in that state. set is a library port covered by unit tests rather than by a stdout oracle.

Layer 2, intermediate files

The command below is the whole check. It exports PERL_USE_UNSAFE_INC=1, sets APP=X, GSFX=X, and BIT2MTX=1 to match mak/mkX.sh, and runs make -f mak/uhun.make ROOT=. xlx once per seed. Expected names include metadict.txt, sfxtags.hpl, encodingXbit2mtx.hpl, stemgenX.lx2, stemgenX.propsets, genX.lx2, sfxgenX.lx2, and prongenX.lx2, plus the lexicon files those rules read and write (st5dbw3s.lx1, 2addsi.lx1, irreg.lx1, and the rest of the closure). Thirty-three files are on the list.

```
python harness/intermediate_state_diff.py --perl-dir emmorph-perl

python harness/intermediate_state_diff.py --perl-dir emmorph-perl --gen-dir /path/to/snapshots
--skip-build
```

--skip-build compares directories named seed-0 and seed-1 that a previous run already captured. The process exits 0 only when every expected file is present in both snapshots and the normalised digests match. Today the public makefile still calls scripts the upstream author did not publish (humlex.pl, ana_only.pl, ana2gen.pl, and others). Those runs end MISSING, which is a failing exit, and the CI job turns that into a warning until the scripts are recovered. An UNSTABLE line is a different event: the file exists twice and its content changed with the hash seed.

Layer 3, analyses

emmorphpy is imported from --emmorphpy-dir when that checkout is present. EmMorphPy is constructed with task analyze and the transducer path. If the import fails, the same words are sent to hfst-lookup and the cohort is the set of outputs that do not end in +?. A word is a match when the two sets are equal. The process exits 0 only when mismatches and errors are both zero.

```
python harness/extrinsic_eval.py --corpus corpus/generated/fuzzed_forms.txt --fst-a
emmorph-perl/hfst/hu.hfstol --fst-b emmorph-perl/hfst/hu.hfstol --emmorphpy-dir ../emmorphpy
```

Until the Python port emits its own transducer, FST B is the same file as FST A. The job then checks the harness, not the port. Replacing --fst-b with the Python artefact is the week-3 step, not a code change.

4. The word list: external, generated, fuzzed

Prof. Indig named three sources in one sentence: a real corpus, every form the grammar can build, and fuzzing. The repository keeps the second and the third. The first is downloaded by the experimenter and is not committed, because the licences differ.

External corpora

corpus/sources/README.md records three downloads. Hungarian Webcorpus 2.0 (https://hlt.bme.hu/en/resources/webcorpus2) is the recommended list: it is large, contemporary, and already used for Hungarian language modelling. The Szeged Corpus (https://rgai.inf.u-szeged.hu/node/120) is smaller and manually tagged, which makes a mismatch easier to adjudicate. OpenSubtitles-HU, from OPUS (https://opus.nlpl.eu/OpenSubtitles/hu&en/v2018/OpenSubtitles), adds conversational inflection that newswire under-represents.

Generated paradigms

corpus/build_paradigm_forms.py reads a Humor lexicon. The default file in the Perl tree is src/st5dbw2s.lx1. The lemma is the text before the category bracket, with compounding plus-signs removed. Nouns (FN) are combined with [/N][Nom], [/N][Acc], [/N][Dat], and [/N][Pl][Nom]. Verbs (IGE) are combined with [/V][Inf] and a present third-person form. Adjectives (MN) are combined with a positive and a comparative tag. hfst-lookup -g receives one analysis string per pair. Surfaces are deduplicated and sorted. --limit stops after a chosen number of lemmas so a first run need not expand the whole lexicon.

```
python corpus/build_paradigm_forms.py --fst emmorph-perl/hfst/hu.hfstol --lexicon
emmorph-perl/src/st5dbw2s.lx1 --out corpus/generated/paradigm_forms.txt --limit 100
```

Fuzzed forms

corpus/fuzz_corpus.py does not call the transducer. It builds stems from onsets, nuclei, and codas that occur at the start of Hungarian words, then appends a short suffix chain. Vowel harmony is the constraint that keeps the strings language-like. A back stem such as alma takes -ban, -nak, -hoz. A front rounded stem such as könyv takes -ben, -nek, -höz. i and í do not by themselves flip a back stem to the front class. --seed fixes the random stream. The same command rewrites the same file.

```
python corpus/fuzz_corpus.py -n 20 --seed 42 -o corpus/generated/fuzzed_forms.txt
```

The committed sample uses -n 1000 and --seed 42. One thousand strings are enough to exercise lookup. They are not a substitute for Webcorpus, and they are not claimed to be grammatical Hungarian. They are claimed to obey onset shape and vowel harmony, and to be regenerable.

5. Three weeks, 18 October to 8 November

The v0.1 window runs from 18 October to 8 November 2026. The three additions fit that window as three weeks. Week 1 is this scaffolding. Week 2 is the first honest layer-2 report. Week 3 is the first extrinsic comparison that has a Python transducer on one side.

Week	Dates	Work

1. Scaffolding	18–24 October 2026	Land the three harnesses, the corpus builders, and this addendum. Leaf ports already in the differential harness stay green.
2. Layer 2	25–31 October 2026	Apply the Perl patches, run xlx under both hash seeds, and separate STABLE files from UNSTABLE producers. Missing scripts stay marked MISSING until they are recovered.
3. Layer 3	1–8 November 2026	Build FST A from the Perl tree, build FST B from the port, and compare a fuzzed list plus a slice of Hungarian Webcorpus 2.0.

Table 2. The three weeks inside the v0.1 window. Dates are inclusive.

Week 2 will not turn the missing scripts into a green build. The report it owes is a table of STABLE, UNSTABLE, and MISSING, with the producer of each UNSTABLE file named from the makefile rule. That table is the input to the decision about which withheld script to reconstruct first. Week 3 depends on a transducer, so it can start from the published hu.hfstol inside emmorphpy while FST B is still a copy of FST A, and switch --fst-b when the port emits its own file.

Bi-weekly notes to Prof. Indig stay on 1 November and 15 November. The 1 November note should attach the layer-2 table if week 2 has finished, and should say plainly which files are still MISSING because the upstream tree does not contain the script.

6. The longer roadmap

The email separates the dissertation-scale port from two later pieces of engineering. They are sequential. Phase 2 is pointless if phase 1 does not emit the same transducer. Phase 3 is pointless if emmorphpy is not the API we intend to keep.

Phase 1, now

Port the Perl compilation pipeline module by module. The acceptance test is a transducer that is byte-identical to the one the Perl pipeline writes. emmorphpy is not modified. It is pointed at one file or the other. Layers 1 and 2 are how we get there. Layer 3, with FST B still equal to FST A, is the dress rehearsal.

Phase 2, one maintained stack

Bring emmorphpy into this repository, or into a sister repository that is versioned and released with it. Today the compiler lives in nytud/emmorph, the lookup wrapper lives in nytud/emmorphpy, and the port lives here. A bug in the tag set has to be fixed in three places before a user sees it. Phase 2 makes the compiler, the transducer, and the wrapper one maintained unit. The public API stays EmMorphPy.analyze, because that is the API the email recommended.

Phase 3, lookup without a process

emmorphpy today starts hfst-lookup with subprocess.Popen and speaks to it through pipes. That design has two costs, and they do not overlap. The first is process startup: the operating system loads the binary and the binary loads the transducer. Paid once per process, it still dominates a short request, and a service that opens a new process per sentence pays it every time. The second is the pipe. Each word is encoded, written to stdin, and read back from stdout, with a round trip in between. A list of a million types pays that cost a million times even if the process stays open.

libhfst already performs the optimised lookup in process for C++ callers. The Python API does not expose that entry point, which is why emmorphpy shells out. Phase 3 adds a binding, with cffi or with pybind11, that keeps the transducer mapped and returns the cohort as a function result. emmorphpy then calls the binding instead of Popen. Startup is paid when the object is constructed. The pipe disappears. The analyses do not change, which is why layer 3 remains the regression test for the binding.

Neither binding is chosen yet. cffi matches a stable C ABI and is easier to ship. pybind11 is the shorter path if the lookup we need is a C++ method without a C wrapper. The choice waits on a reading of the installed libhfst headers, not on this addendum.

7. The thirty-three files and the CI jobs

Layer 2 does not hash whatever happens to be in gen/. It hashes a fixed list derived from the xlx closure in mak/uhun.make, with the same APP=X, GSFX=X, and BIT2MTX=1 that mak/mkX.sh exports. The encoding file is therefore encodingXbit2mtx.hpl. A bare encodingX.hpl is not on the list, because that name is what the makefile writes only when BIT2MTX is unset. timestamp is also omitted: its recipe prints the local clock and would fail the gate on every run.

File	File
metadict.txt	stemgenX.propsets
encodingXbit2mtx.hpl	stemgenX.avs
sfxtags.hpl	stems2addgenX.lx2
stemalt1.pl	stems2addgenX.propsets
sfxalt1.pl	stems2addgenX.avs
st5dbw3s1.lx1	prongenX.lx2
st5dbw3s.lx1	prongenX.propsets
st5dbw3si.lx1	prongenX.avs
2adds.lx1	genX.lx2
2adds1.lx1	genX.propsets
2addsi.lx1	genX.avs
irreg.lx1	sfxgenX.lx2
Xsu1.lx1	sfxgenX.propsets
Xsu.lx1	sfxgenX.avs
Xs1.lx1	sfxX.lx2
Xsi.lx1	sfxX.propsets
stemgenX.lx2	

Table 3. The thirty-three names intermediate_state_diff.py hashes. A name is STABLE when both seeds produced it and the normalised sha256 agrees, UNSTABLE when both exist and the digest differs, and MISSING when either seed failed to write it.

The public tree does not contain every producer those rules call. humlex.pl, ana_only.pl, strdiff.pl, stat2lex.pl, sfxtaglst.pl, prd2stat.pl, lex2prd.pl, genfix.pl, and ana2gen.pl are referenced and not shipped, and src/dersfx.lst is absent as well. Until those gaps close, a full xlx run ends with MISSING rows. That exit is 1. It is the correct exit: a build that did not write the file has not shown the file to be stable. CI records the same fact as a warning, with || true, so the missing upstream scripts do not paint the workflow red while the warning stays visible in the log.

The workflow keeps the four original jobs and adds two. intermediate-state needs reproducibility-test. It applies scripts/perl_patches and then runs python harness/intermediate_state_diff.py --perl-dir emmorph-perl. extrinsic-eval needs differential-harness. It clones nytud/emmorphpy beside the checkout, runs bash mak/mkemmorph.sh inside emmorph-perl, writes a 1000-word fuzzed list with seed 42, and calls extrinsic_eval.py with that list and with FST A on both sides. The second warning is there because FST B does not exist yet. Pointing both arguments at hu.hfstol checks the harness. It does not claim the port has a transducer.

What a normalised digest ignores

A line such as %mcat=('b','2','a','1'); and the line %mcat=('a','1','b','2'); are the same assignment. normalize_line sorts the pairs by key, so both become %mcat=('a','1','b','2'); before sha256. Two builds that differ only in the order Perl happened to emit a hash are STABLE. Two builds that differ in a value, a missing key, or any line that is not a

hash assignment are UNSTABLE. Binary files that are not valid UTF-8 are hashed as raw bytes, because there is no line order to repair.

The first execution of the harness on this working copy, on 18 October 2026, did not reach Perl. make is not on the PATH of the machine that holds the checkout, so both seeds exited 127 and all thirty-three names were MISSING. The process exited 1. No gen/ directory was left behind: the script moves any existing gen/ aside and restores it. The extrinsic command exited 2, because emmorph-perl/hfst/hu.hfstol is not in the tree. That file is produced by mak/mkemmorph.sh, which the CI job runs and this machine cannot. The fuzzed list the extrinsic job will read is already written: one thousand forms from corpus/fuzz_corpus.py with --seed 42.

A synthetic pair is enough to see the three labels without a makefile. Identical text is STABLE. The same path missing on one side is MISSING. Two different texts are UNSTABLE. The unit tests under tests/test_indig_layers.py assert those three outcomes, the lemma split on src/st5dbw2s.lx1 (the text before the category bracket, plus-signs dropped, a leading asterisk stripped), and the fuzzer's contract: twenty forms at seed 42 are sorted, unique, and identical on a second draw. alma is back, könyv is front-rounded, szék is front. Back stems take back suffixes. Front stems take front suffixes.

Drop-in comparison, which the email asks for in the same sentence as the state check, is the other half of the method. A Python module replaces one Perl script in the pipeline, the rest of the pipeline stays Perl, and the gen/ files downstream of that script are diffed immediately. Layer 2 is that diff with no replacement yet, so the two seeds measure the Perl pipeline against itself. When a module is swapped, the same command is the regression: seed 0 of the mixed pipeline against seed 0 of the pure Perl pipeline, file by file.

8. References

Indig, Balázs (2026). Email to Farzaneh Velaeishekan, 17 October. The three quotations in section 1 are taken from that letter.

Nemeskey, Dávid Márk (2020). Natural Language Processing Methods for Language Modeling. PhD thesis, Eötvös Loránd University. Source of Hungarian Webcorpus 2.0.

Novák, Attila (2003). Milyen a jó Humor? In Magyar Számítógépes Nyelvészeti Konferencia (MSZNY 2003), pp. 138–145. Szeged.

Novák, Attila (2014). A New Form of Humor. Mapping Constraint-Based Computational Morphologies to a Finite-State Representation. In Proceedings of LREC 2014, pp. 1068–1073. Reykjavík.

Novák, Attila, Borbála Siklósi, and Csaba Oravecz (2016). A New Integrated Open-source Morphological Analyzer for Hungarian. In Proceedings of LREC 2016, pp. 1315–1322. Portorož.

Lison, Pierre and Jörg Tiedemann (2016). OpenSubtitles2016: Extracting Large Parallel Corpora from Movie and TV Subtitles. In Proceedings of LREC 2016. Source of the OpenSubtitles-HU list.

nytud/emmorph, https://github.com/nytud/emmorph. The Perl pipeline and the lexicon. nytud/emmorphpy, https://github.com/nytud/emmorphpy. The lookup wrapper used by layer 3.

End of addendum v0.2. The v0.1 baseline report is unchanged.